

Московский авиационный институт
(национальный исследовательский университет)

Факультет Компьютерные науки и прикладная математика

Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Дискретный анализ»

Студент: Савельев Артем
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 11.04.2026
Оценка:
Подпись:

Москва, 2026

1 Условие задачи

Для реализации словаря из предыдущей лабораторной работы, необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

Результатом лабораторной работы является отчёт, состоящий из:

- Дневника выполнения работы, в котором отражено что и когда делалось, какие средства использовались и какие результаты были достигнуты на каждом шаге.
- Выводов о найденных недочётах.
- Сравнения работы исправленной программы с предыдущей версией.
- Общих выводов о выполнении лабораторной работы, полученном опыте.

Минимальный набор используемых средств должен содержать утилиту для профилирования (в данной работе использован `gprof` / `Samplify`) и библиотеку/утилиту для поиска утечек памяти (в данной работе использована встроенная утилита `leaks` на macOS).

2 Исходный код ключевых алгоритмов

В ходе оптимизации рекурсивные методы `insert` и `remove` были заменены на итеративные с использованием статического массива для отслеживания пути от корня к листу. Это позволило избежать накладных расходов на вызовы функций, а также наконец воспользоваться `std::move` для предотвращения копирований при вставке новых узлов. Ниже представлен код:

```
1 bool insert(Key key, Value value) {
2     if (!root_) {
3         root_ = new Node(std::move(key), std::move(value));
4         ++size_;
5         return true;
6     }
7
8     Node* path[128];
9     int path_len = 0;
10    Node* curr = root_;
11
12    while (curr) {
13        path[path_len++] = curr;
14        if (key < curr->key) {
15            if (!curr->left) {
16                curr->left = new Node(std::move(key),
17                                     std::move(value));
18                path[path_len++] = curr->left;
19                ++size_;
20                break;
21            }
22            curr = curr->left;
23        } else if (key > curr->key) {
24            if (!curr->right) {
25                curr->right = new Node(std::move(key),
26                                      std::move(value));
27                path[path_len++] = curr->right;
28                ++size_;
29                break;
30            }
31            curr = curr->right;
32        } else {
33            return false;
34        }
35    }
36
37    for (int i = path_len - 2; i >= 0; --i) {
38        Node* node = path[i];
39        Node* new_subtree = balance(node);
40
41        if (new_subtree != node) {
42            if (i == 0) {
43                root_ = new_subtree;
44            } else {
45                Node* parent = path[i - 1];
46                if (parent->left == node) parent->left = new_subtree;
```

```

45         else parent->right = new_subtree;
46     }
47 }
48 }
49 return true;
50
51
52 bool remove(Key key) {
53     if (!root_) return false;
54
55     Node* path[128];
56     int path_len = 0;
57
58     Node* curr = root_;
59
60     bool found = false;
61     Node* target = nullptr;
62     while (curr) {
63         path[path_len++] = curr;
64
65         if (key < curr->key) {
66             curr = curr->left;
67         } else if (key > curr->key) {
68             curr = curr->right;
69         } else {
70             found = true;
71             target = curr;
72             break;
73         }
74     }
75     if (!found) return false;
76
77     Node* parent = (path_len >= 2) ? path[path_len - 2] : nullptr;
78
79     if (!target->left || !target->right) {
80         Node* child = target->left ? target->left : target->right;
81
82         if (parent == nullptr) {
83             root_ = child;
84         } else if (parent->left == target) {
85             parent->left = child;
86         } else {
87             parent->right = child;
88         }
89         delete target;
90     } else {
91         Node* succ_parent = target;
92         Node* succ = target->right;
93         while (succ->left) {
94             succ_parent = succ;
95             succ = succ->left;
96         }
97

```

```

98     target->key = std::move(succ->key);
99     target->value = std::move(succ->value);
100
101     if (succ_parent->left == succ) {
102         succ_parent->left = succ->right;
103     } else {
104         succ_parent->right = succ->right;
105     }
106     delete succ;
107 }
108
109 --size_;
110
111 for (int i = path_len - 2; i >= 0; --i) {
112     Node* node = path[i];
113     Node* new_subtree = balance(node);
114
115     if (new_subtree != node) {
116         if (i == 0) {
117             root_ = new_subtree;
118         } else {
119             Node* p = path[i - 1];
120             if (p->left == node) {
121                 p->left = new_subtree;
122             } else {
123                 p->right = new_subtree;
124             }
125         }
126     }
127 }
128
129 return true;
130 }
131 }

```

3 Дневник выполнения работы и анализ недочётов

3.1 Дневник выполнения работы

1. **День 1: Базовая проверка памяти.** Первоначальная реализация AVL-дерева была протестирована с помощью утилиты `leaks`. Благодаря корректной реализации рекурсивного метода `destroy()` в деструкторе, утечек памяти обнаружено не было (0 leaks for 0 total leaked bytes).

Посмотрим на результаты неоптимизированных бенчмарков:

```
1 artems@MacBook-Air-artem-3 lab2 % leaks --atExit --
  ./benchmark_gprof_non_optim tests/01
2 benchmark_gprof(40739) MallocStackLogging: could not tag
  MSL-related memory as no_footprint, so those pages will be
  included in process footprint - (null)
3 benchmark_gprof(40739) MallocStackLogging: recording malloc and
  VM allocation stacks using lite mode
4 Loading data into memory... Done. Loaded 643408 operations.
5
6 Running std::map... Done.
7 Running avl::AVL... Done.
8
9 === BENCHMARK RESULTS ===
10 Total Operations : 643408
11 -----
12 std::map (RB)      :      790 ms | Mismatches: 0
13 avl::AVL           :     1379 ms | Mismatches: 0
14 -----
15 SUCCESS: Both trees produced 100% correct results.
16 Process 40739 is not debuggable. Due to security restrictions,
  leaks can only show or save contents of readonly memory of
  restricted processes.
17
18 Process:          benchmark_gprof [40739]
19 Path:             /Users/USER/Desktop/*/benchmark_gprof
20 Load Address:    0x102f58000
21 Identifier:       benchmark_gprof
22 Version:          0
23 Code Type:        ARM64
24 Platform:         macOS
25 Parent Process:   leaks [40738]
26
27 Date/Time:         2026-04-12 18:20:57.219 +0300
28 Launch Time:       2026-04-12 18:20:52.600 +0300
29 OS Version:        macOS 14.6.1 (23G93)
30 Report Version:    7
31 Analysis Tool:     /usr/bin/leaks
32
33 Physical footprint:      263.8M
34 Physical footprint (peak): 547.4M
35 Idle exit:              untracked
36 ----
37
38 leaks Report Version: 4.0, multi-line stacks
```

```

39 Process 40739: 186 nodes malloced for 15 KB
40 Process 40739: 0 leaks for 0 total leaked bytes.

```

2. **День 2: Профилирование времени.** Запуск профилировщика (Samplify) показал, что программа работает медленнее эталонного `std::map`. Анализ графа вызовов выявил, что около 29% процессорного времени тратится на файловый ввод (`std::istream`) и около 13% на самую вставку в дерево. Также замечено значительное время, затрачиваемое на копирование ключей (`std::string`).
3. **День 3: Оптимизация (Избавление от рекурсии).** Было принято решение отказаться от рекурсивного метода `insert`, так как каждый рекурсивный вызов создает новый фрейм стека. Написана итеративная версия вставки с использованием локального массива `Node* path[128]` для хранения пути. Ключи и значения начали передаваться через `std::move`.
4. **День 4: Финальные замеры.** Проведено сравнительное тестирование неоптимизированной и оптимизированной версий программы под контролем `leaks`.

```

1  artems@MacBook-Air-artem-3 lab2 % g++ -std=c++17 -pg -O2 -g -Wall
   -Wextra \
2      benchmark.cpp -o benchmark_gprof
3  artems@MacBook-Air-artem-3 lab2 % leaks --atExit --
   ./benchmark_gprof_optim tests/01
4  benchmark_gprof(41393) MallocStackLogging: could not tag
   MSL-related memory as no_footprint, so those pages will be
   included in process footprint - (null)
5  benchmark_gprof(41393) MallocStackLogging: recording malloc and
   VM allocation stacks using lite mode
6  Loading data into memory... Done. Loaded 643408 operations.
7
8  Running std::map... Done.
9  Running avl::AVL... Done.
10
11 === BENCHMARK RESULTS ===
12 Total Operations : 643408
13 -----
14 std::map (RB)      :      820 ms | Mismatches: 0
15 avl::AVL           :      782 ms | Mismatches: 0
16 -----
17 SUCCESS: Both trees produced 100% correct results.
18 Process 41393 is not debuggable. Due to security restrictions,
   leaks can only show or save contents of readonly memory of
   restricted processes.
19
20 Process:           benchmark_gprof [41393]
21 Path:              /Users/USER/Desktop/*/benchmark_gprof
22 Load Address:     0x104df0000
23 Identifier:        benchmark_gprof
24 Version:           0
25 Code Type:         ARM64
26 Platform:          macOS
27 Parent Process:    leaks [41392]
28
29 Date/Time:         2026-04-12 18:24:04.060 +0300

```

```

30 Launch Time:      2026-04-12 18:24:00.439 +0300
31 OS Version:      macOS 14.6.1 (23G93)
32 Report Version:   7
33 Analysis Tool:    /usr/bin/leaks
34
35 Physical footprint:      9.8M
36 Physical footprint (peak): 296.0M
37 Idle exit:              untracked
38 ----
39
40 leaks Report Version: 4.0, multi-line stacks
41 Process 41393: 186 nodes malloced for 15 KB
42 Process 41393: 0 leaks for 0 total leaked bytes.

```

3.2 Выводы о найденных недочётах

В ходе исследования были выявлены следующие архитектурные недочёты первой версии:

- **Передача параметров по значению в рекурсии:** При передаче строк по значению (`std::string`) на каждом шаге рекурсии происходило выделение памяти под копию строки. Использование `const Key&` или `std::move` критически важно для производительности.
- **Накладные расходы стека вызовов:** Рекурсивная вставка порождала глубокий стек. Это не только замедляло исполнение из-за сохранения регистров процессора, но и усложняло кэширование.

3.3 Общие выводы об опыте

Лабораторная работа наглядно продемонстрировала важность использования профилировщиков.

4 Результаты бенчмарков и сравнение версий

4.1 Влияние «эффекта наблюдателя» (Тестирование под leaks)

При запуске тестов под утилитой проверки памяти `leaks` (с включенным `MallocStackLogging`) время работы программы значительно увеличивается. Это происходит из-за перехвата каждого вызова `new` для создания узлов дерева.

Алгоритм	Рекурсия	Итератив	Память (пик)
<code>std::map</code>	790 ms	820 ms	296.2 MB
<code>avl::AVL</code>	1379 ms	782 ms	299.7 MB
Утечки	0	0	—

Таблица 1: Производительность и потребление памяти под `leaks` (643 408 операций)

Несмотря на искажение времени, виден качественный скачок: итеративная версия требует почти в два раза меньше времени с этой утилитой.

4.2 Нативные замеры производительности

Для получения «чистых» метрик программа была скомпилирована с флагом `-O2` и запущена без инструментов отладки памяти.

Алгоритм	Время (рекурс.)	Время (итерат.)	RSS пик (рекурс.)	RSS пик (и
<code>std::map</code> (эталон)	507 ms	502 ms	250.2 MB	250.2 M
<code>avl::AVL</code> (наша реализация)	477 ms	472 ms	248.5 MB	247.4 M

Таблица 2: Сравнение времени выполнения и пикового потребления памяти (RSS) при 643 408 операциях

Оптимизированная итеративная версия AVL-дерева оказалась быстрее `std::map` примерно на 5 миллисекунд на большом тесте по сравнению с рекурсивной.

4.3 Анализ стека вызовов (Flame Graph)

Для визуального анализа были построены `**flame graphs**` с помощью утилиты `samply` (или Xcode Instruments → Time Profiler).

На рекурсивной версии (Рис. 2) явно видна глубокая вложенность вызовов функции `insert` и `remove`. Итеративная версия (Рис. 1) имеет практически плоский профиль, который объясняется уходом от рекурсии.

